Nature-Inspired Artificial Intelligence

2025/2026

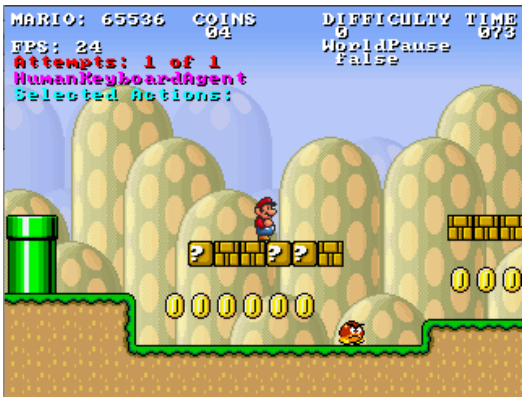
"It's-a Me, Mario:"

Evolving the Ultimate Italian Plumber

Nuno Lourenço

João Macedo

March 5, 2026



1 Introduction

We present here the practical project, part of the students' evaluation process of the Nature-Inspired AI course of the University of Coimbra. This work is to be done autonomously by a group of **two** students. The deadline for delivering the work is **05 of May** via Inforestudante.

The quality of your work will be judged as a function of the value of the technical work, the written description, and the **public defence**. All sources used to perform the work (including the code) must be clearly identified. The document may be written in Portuguese or in English, using a word processor of your choice[1]. The written report is limited to **10** pages long, but in special, justified, cases (e.g., the need of presenting many images and/or tables), that number may be increased accordingly. The document should be well structured, including a general introduction, a description of the problem, the experimental setup, the analysis of the results, and a conclusion. The report should follow the Springer LNCS format. The Latex and Word templates are available in the Support Material of the course. The final mark will be given to each member of the group individually.

To do the work the student may consult any source he/she wants. Nevertheless, plagiarism will not be allowed and, if detected, it will imply failing the course. While doing the work and when submitting it, you should pay particular attention to the following aspects (whose relative importance depends on the type of work done):

2 Problem Statement

Video games serve as sophisticated **testbeds** for Artificial Intelligence (AI) research because they mimic real-world complexities within a controlled and reproducible environment. Unlike traditional optimization problems, games require the coordination of atomic behaviors into complex sequences to navigate high-dimensional state spaces.

The use of games for AI research is motivated by several key factors:

- **Human-Level Skill Relevance**: The skills required to play platform games, such as spatial reasoning and real-time decision-making, are analogous to those humans possess and are relevant to broader AI goals.
- **Computational Efficiency**: Virtual simulators allow agents to be evaluated at speeds significantly faster than real-time, enabling thousands of evolutionary iterations in a short period.

[1] We strongly suggest the use of LaTeX.

- **Interpretability**: While many AI methods like Neural Networks are often "black boxes," symbolic approaches like **Genetic Programming (GP)** provide human-readable source code. This allows researchers to understand the agent's decision-making process and extract useful gameplay strategies.

The objective of this assignment is to design and evolve an autonomous controller for **Super Mario**. We are going to use the framework developed by Julian Togelius et al. for the Mario AI Championship [2].

The framework presented in [1] utilizes a customized iteration of *Infinite Mario Bros*, a public-domain adaptation of Nintendo's *Super Mario Bros* developed by Markus Persson. The original version of this clone is accessible via web browsers, with its Java-based source code openly available for modification.

2.1 Gameplay Mechanics and Action Space

The core gameplay is defined by a 2D side-scrolling physics engine where the agent must manage spatial navigation and state transitions.

2.1.1 Agent Action Space

The controller interacts with the environment through a discrete set of actions that determine Mario's trajectory and offensive capabilities:

- **Lateral Movement:** Moving left or right to traverse the level or position the agent.
- **Jump:** A vertical impulse used to clear gaps (pits), reach higher platforms, or perform an "Overhead Attack" by landing on an enemy's hit-box.
- **Sprint/Fire:** Increasing movement velocity (running) or, when in the *Fire* state, discharging a projectile to neutralize enemies at a distance.

2.1.2 State-Dependent Dynamics

Mario interaction with hazards is governed by his current *Power-up State*. These states act as a health buffer and grant additional environmental interactions:

[2] http://julian.togelius.com/mariocompetition2009/advanced.php

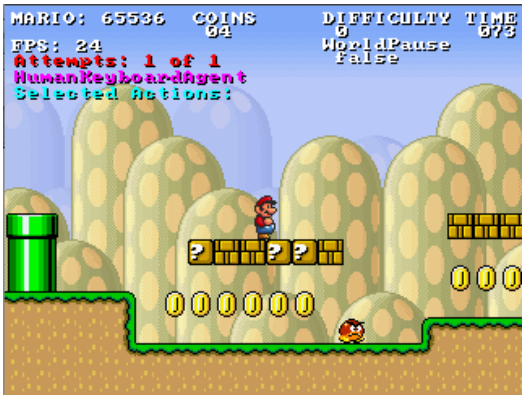


Figure 1: Super Mario Game Environment

- **Small:** The baseline state. Any collision with an enemy or hazard results in an immediate *Loss of Life*.
- **Big:** Acquired via *Mushrooms*. This state allows Mario to break brick blocks from below and provides a safety buffer: contact with an enemy reverts Mario to the **Small** state rather than ending the game.
- **Fire:** Acquired via *Fire Flowers*. Retains the "Big" state benefits but adds the offensive ability to shoot fireballs. Damage reverts Mario to the **Big** state.

2.1.3 Hazard Interaction and Physics

Success is determined by navigating two primary failure conditions:

1. **Environmental Pits:** Gravity is a constant force; falling into a gap (y-axis depletion) results in an unconditional *Loss of Life*, regardless of state.
2. **Enemy Collisions:**
    - **Vertical Interaction:** Landing on top of most enemies (Goombas, Koopas) neutralizes them.
    - **Horizontal/Lateral Interaction:** Touching an enemy from the side or below triggers the damage hierarchy mentioned above.
    - **Invulnerability:** Certain enemies, such as Piranha Plants, cannot be defeated via jumping and must be avoided or shot.

3 Objective

The primary objective of this project is to design and implement an Evolutionary Algorithm (EA) capable of evolving autonomous controllers for Super Mario. The evolved agents must effectively navigate game environments to achieve specific performance benchmarks.

To accomplish this, the development process must address the following fundamental design components of a Nature-Inspired approach:

- **Genotype Representation:** Defining how the controller's parameters are encoded.
- **Initialization Procedure:** Establishing the starting population of potential solutions.
- **Variation Operators:** Implementing robust crossover and mutation mechanisms to explore the search space.
- **Selection Mechanisms:** Determining both Parent Selection (for reproduction) and Survivor Selection (for population replacement).
- **Fitness Function:** Designing a heuristic to evaluate agent performance.

As established in our theoretical discussions, the **fitness function** is the most critical driver of evolutionary success. Students are expected to iteratively experiment with various fitness formulations.

The ultimate goal is to produce an agent that reliably clears levels while maximizing cumulative rewards. You should consider that your ultimate task should be divided into two distinct developmental stages:

1. **General Completion:** Evolving a controller capable of successfully finishing up to five unique levels.
2. **Combat Optimization:** Refining a controller specifically to maximize the elimination of in-game enemies.

4 Methodology

The methodology is divided into two core components: the search strategy utilized to navigate the solution space and the design of the fitness function used to evaluate agent performance.

4.1 Search Strategy

To evolve an autonomous controller, one of the following two nature-inspired paradigms must be selected:

- **Genetic Programming (GP):** In this approach, the evolutionary process directly operates on symbolic expressions or Python code snippets. The population consists of executable programs that map game observations to controller actions.
- **Neuroevolution:** This approach utilizes an Evolutionary Algorithm to optimize the weights of a Multi-Layer Perceptron (MLP). The topology remains static or evolves to transform input features into motor commands.

4.2 Fitness Function

The fitness function serves as the heuristic measure of an individual's "quality" or "biological fitness." To evaluate the performance of a controller, students must derive a scoring metric using the following sensorial information provided by the base agent's observation state:

- **Binary State Indicators:**
  - `can_jump`: A boolean attribute indicating whether the agent is currently in a state that allows for a jump action.
  - `on_ground`: A boolean indicating if Mario is currently in physical contact with the floor or a platform.
- **Spatial Coordinates:**
  - `mario_floats`: The floating-point representation of Mario's current (x, y) position within the level.
  - `enemies_floats`: A list of the floating-point (x, y) coordinates for all active enemies within the agent's processing radius.
- **Environmental Perception (`level_scene`):** The agent perceives its surroundings through a 22×22 NumPy array, with Mario fixed at the center coordinate $[11, 11]$ (Fig. 2). Each integer value within the array corresponds to a specific entity or environmental property, as defined in Table 1.

Students should use these variable to build Fitness Function(s) to allow the fullfilment of the objectives describe in Section 3.

Value	Entity / Meaning
-11	Soft obstacle (can jump through)
-10	Hard obstacle (cannot pass through)
0	No obstacle or enemy (empty space)
1	Mario
2	Enemy: Goomba
3	Enemy: Goomba (winged)
4	Enemy: Red Koopa
5	Enemy: Red Koopa (winged)
6	Enemy: Green Koopa
7	Enemy: Green Koopa (winged)
8	Enemy: Bullet Bill
9	Enemy: Spiky
10	Enemy: Spiky (winged)
12	Enemy: Piranha Flower
13	Enemy: Shell
14	Item: Mushroom
15	Item: Fire Flower
16	Brick (simple / coin / mushroom / flower)
20	Enemy obstacle (e.g., flower pot, cannon parts)
21	Question brick (with coin or mushroom/flower)
25	Mario weapon projectile (Fireball)
42	Undefined

Table 1: Numerical mapping of the `level_scene` observation grid.

4.3 Action Space and Control Signals

The controller interacts with the environment by returning a discrete action vector at each time step. This vector is an array of five binary integers, where each position corresponds to a specific control signal. A value of 1 activates the action, while 0 keeps it disabled.

The control vector is structured as follows:

```
[backward, forward, crouch, jump, speed/bombs]
```

4.3.1 Action Mapping and Execution

To successfully navigate the environment, the evolved controller must learn to output the correct combination of signals. For example:

- **Right Traversal:** Returning `[0, 1, 0, 0, 0]` moves Mario forward.

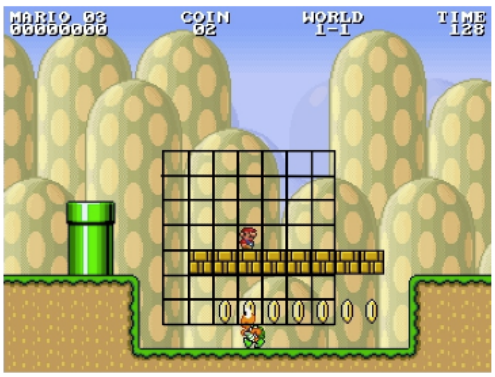


Figure 2: Level Scene is a 22x22 numpy array containing the elements of the level around the agent (Mario is at [11, 11]). Adapted from [1].

- **Jump Backward:** Returning `[1, 0, 0, 1, 0]` combines the first and fourth indices.

4.3.2 Implementation Details

This control logic must be implemented within the `act()` function of the specific agent classes located in the `agents` module. Depending on the chosen **Search Strategy**, the implementation will reside in one of the following:

- **Genetic Programming:** The logic is handled by the `CodeAgent` class, where the evolved program is interpreted to return the action vector.
- **NeuroEvolution:** The logic is handled by the `MLPAgent` class, where the neural network's forward pass determines the output vector based on the optimized weights.

Regardless of the approach, the `act()` function serves as the interface between the evolutionary search space and the game engine's physics.

5 Evaluation and Comparative Analysis

To validate the performance of the chosen evolutionary approach, a comparative study must be conducted. The main goal is to show that the evolved controller outperforms a baseline of purelely random decision-making.

5.1 Experimental Setup

Students must select a single implementation, either the `CodeAgent` (Genetic Programming) or the `MLPAgent` (NeuroEvolution), and subject it to the following experimentation:

1. **Evolved Agent:** The best-performing individual (the "champion") produced by the evolutionary search.
2. **Random Search Baseline:** A controller of the same class (`CodeAgent` or `MLPAgent`) where the parameters (code or weights) are generated randomly rather than through evolutionary selection.

5.2 Performance Metrics

Both agents must be evaluated across the 3 different levels. For each level, the following metrics should be recorded and compared:

- **Completion Rate:** The percentage of attempts that finish the level.
- **Traversal Efficiency:** The maximum distance (`mario_floats.x`) reached before death or the expiration of time.
- **Reward Acquisition:** Total enemies defeated and/or coins collected, derived from the `Observation` object during the run.

5.3 Statistical Significance

As evolutionary algorithms are stochastic, a single run is insufficient. It is recommended to run the evaluation over multiple seeds ($N \geq 5$) and report the mean and standard deviation for each metric.

Metric	**Random Search (Avg ± STD)**	**Evolved Agent (Avg ± STD)**	**Improvement (%)**
Levels Cleared	X	Y	$+Z\%$
Max Distance	X	Y	$+Z\%$
Enemies Killed	X	Y	$+Z\%$

Table 2: Comparison of Evolved Controller vs. Random Search Baseline.

5.4 Discussion of Findings

The results should analyze the behavior of the random agent (e.g., "The random MLP frequently performed irrelevant jumps") versus the evolved agent (e.g., "The evolved GP script successfully prioritized the `forward` signal while reacting to enemies in the `level_scene`").

6 Conclusion

A few short comments. First, the control of the progression of your work will be done during the classes (T and PL). Moreover, you can discuss eventual problems by presenting yourself during office hours. Second, the projects reflect for the most part your actual knowledge. Third, we try to balance the difficulty of all the work, but we are aware that this is not an easy task and it is somehow a subjective matter. Fourth, we try to ask a workload compatible with the value of the work for the final mark.

Methodological issues, like the statistical background, were elucidated during the previous lectures. You may use the statistical tool you feel at ease with, including the Python code that was provided. Finally, even if this is a work that asks you to do simulations and analyze the results, i.e., it has a practical flavor, there is however a theory behind the work, and you are advised to consult the necessary literature.

Good luck!

A Project Implementation and Experimentation Checklist

This section provides a brief checklist aimed at minimizing the probability of serious gaps in the project and report. It is important to emphasize, however, that this checklist does not replace the validation of the choices made, which should occur during the Laboratory Practical classes, nor does it guarantee a positive final grade.

A.1 Programming and Technical Implementation

A.1.1 Controller and Action Space

- **Strategy Selection:** Choose exactly one paradigm: *Genetic Programming* (`CodeAgent`) or *Neuroevolution* (`MLPAgent`).
- **The `act()` Function:** Ensure the function returns a list of five binary integers: `[backward, forward, crouch, jump, speed/bombs]`.
- **Sensory Integration:** Are you using the environment's information in the controller inputs.

A.1.2 Evolutionary Engine Design

- **Population Initialization:** Implement a procedure to generate an initial diverse population of programs or weight matrices.
- **Selection Mechanisms:**
  - Implement a *Parent Selection* method (e.g., Tournament or Roulette Wheel).
  - Implement a *Survivor Selection* method (e.g., Elitism) to preserve the best individual across generations.
- **Variation Operators:**
  - **Crossover:** Define how genetic material is exchanged between two high-performing candidates.
  - **Mutation:** Implement stochastic alterations to parameters to explore the search space and prevent local optima.

A.1.3 Fitness Function Formulation

- **Stage 1 (General Completion):** Design a fitness score that allows the completion of levels.
- **Stage 2 (Combat Optimization):** Include a mechanism to reward the neutralization of enemies based on the `level_scene` mapping.

A.2 Experimentation and Evaluation

A.2.1 Comparative Baseline

- **Random Search Setup:** Implement a control group using the same agent class (`CodeAgent` or `MLPAgent`) but with randomized parameters.
- **Benchmark Evaluation:** Run the Random Agent and the Evolved Agent through the same levels to measure the gain provided by the Evolutionary Algorithm.

A.2.2 Statistical Reliability

- **Independent Trials ($N == 5$):** Conduct the evolutionary process across at least five different random seeds to account for stochasticity.
- **Quantitative Reporting:** Calculate and report the Mean and Standard Deviation ($Avg \pm STD$) for:
  - Completion Rate (%).
  - Maximum Distance Traversed.
  - Total Enemy Eliminations.

A.2.3 Final Assessment

- **Multi-Level Testing:** Evaluate the "Champion" agent on at least three distinct levels.
- **Generalization Analysis:** Observe if the controller evolved on a specific level can successfully navigate unseen level geometries.

A.3 Report and Submission (Springer LNCS)

- **Formatting:** Adhere to the Springer LNCS template and the 10-page limit.

- **Data Visualization:** Include a *Fitness Convergence Curve* and the *Comparison Table* against the Random Baseline.
- **Discussion:** Provide a qualitative analysis of the agent's behavior, identifying specific obstacles or enemies that the evolved strategy struggled to overcome.

B Genetic Programming Agent

We provide an example of a simple Random Search using Genetic Programming and the DEAP framework. The code we provide allows the construction of Mario agents with a very limited vocabulary. If you look at the **Primitive Definitions** in the code, Mario only knows how to do two things: move `RIGHT` and `JUMP`, and he only knows two states: `on_ground` and `can_jump`.

He is essentially a blind kangaroo. He will hop forward until he hits a wall he cannot jump over or walks directly into a Goomba.

To improve the agent, you should expand the primitive set to generate smarter behaviors.

B.1 Phase 1: Motor Skills and Basic Senses

B.1.1 Challenge 1: Unlocking the Controller

Sometimes, moving right is not enough. Mario might need to back up, or use his speed button to make longer jumps.

- **Task:** Add the remaining controller buttons to the terminal set.
- **Hint:** Look at the `pset.addTerminal` lines. You need to add `Mario.KEY_LEFT`, `Mario.KEY_DOWN`, and `Mario.KEY_SPEED` of type `Key`.

B.1.2 Challenge 2: The Gift of Sight (Enemies)

Mario does not know enemies exist! The environment provides an `enemies` grid (a numpy array where Mario is in the center), but our grammar doesn't have a way to read it yet.

- **Task:** Introduce the ability to check for enemies.
- **Steps:**
  1. Define a python function `str_check_enemy(pos_x, pos_y, comp, enemy_type)` that returns a string of python code (e.g., `"enemies[(11+y), (11+x)] == 80"`).
  2. Add this function as a Primitive to your `pset` returning a `Condition`.
  3. Add the necessary Terminals: Positions (`-1`, `0`, `1`), Comparators (`==`, `!=`), and Enemy Types (like `Sprite.KIND_GOOMBA`).

B.2 Phase 2: Complex Decision Making

B.2.1 Challenge 3: Plan B (If-Then-Else)

Right now, Mario can only use `If-Then` logic (e.g., *If I can jump, then jump*). But what if he needs a fallback plan? (e.g., *If there is a Goomba ahead, jump; ELSE, run right*).

- **Task:** Create a new string builder function for `if_then_else` and add it to the Primitive Set.
- **Hint:** This primitive will take *three* arguments: a `Condition`, an `Expr` (True branch), and another `Expr` (False branch).

B.2.2 Challenge 4: Understanding the Terrain

Mario is getting stuck on pipes and walls because he can't see the landscape. The environment provides a `landscape` array.

- **Task:** Similar to the enemy check, add a `str_check_landscape` primitive and the corresponding landscape terminals (e.g., `16` for bricks, `-10` for hard blocks).

C MLP Agent

We provide a baseline example of a naive Random Search evolving the weights of a Multi-Layer Perceptron (MLP) using PyTorch. In this approach, Mario's brain is a Neural Network. If you look at the `MLPAgent` and `MLP` classes, you will see that Mario's decision-making is defined by a continuous vector of weights and biases.

This approach presents two challenges. First, the search space is astronomically large. The current network takes in 488 inputs (mostly the 22×22 `level_scene`) and processes them through two hidden layers. If you extract the `param_vector`, you will find it contains roughly **16,000 individual parameters**! Throwing random darts in a 16,000-dimensional space will never yield a winning strategy.

Your objective is to upgrade both the **Optimizer** (the search algorithm) and the **Architecture** (Mario's brain) to conquer this environment.

C.1 Phase 1: Upgrading the Optimizer

Your current optimizer is a pure random search. Because the starting `param_vector` is never updated inside the loop, the algorithm never actually "walks" toward a better solution. To navigate this complex landscape, you must replace this naive search with a structured optimization strategy.

C.1.1 Challenge 1: Selecting an Evolutionary Algorithm

To navigate a 16,000-dimensional space, your algorithm must have a strategy to step toward better solutions and remember its successes.

- **Task:** Select an Evolutionary Algorithm from the ones discussed in class (e.g. $1+\lambda$ ES, Genetic Algorithms, or $\mu+\lambda$ Evolution Strategies) and implement it to replace the pure random search.

- **Elitism:** Regardless of the algorithm you choose, you should implement **Elitism**. Ensure the absolute best agent from the current generation is copied directly into the next generation *without* mutation. If you add random noise to every single agent, Mario will inevitably forget how to walk.

C.2 Phase 2: Upgrading the Architecture (Feature Engineering)

A neural network is only as good as the data you feed it. The provided `MLPAgent` requires significant engineering to become viable.

C.2.1 Challenge 2: The Gift of Sight (Processing Enemies)

Read the `act()` method in your starter code. You will see the comment: *"enemies_floats turned out to be variable, ignoring for now."* Mario is currently running entirely blind to enemies! Neural networks require fixed-size inputs, meaning you cannot pass a variable-length list of enemies directly into the MLP.

- **Task:** Reintroduce enemy detection into the `input_tensor`.
- **Hint:** You must engineer a fixed-size representation. You could map the enemies onto a fixed 22×22 grid (similar to the `level_scene`), or sort the enemies by distance and only pass the (x, y) coordinates of the **three closest enemies**.

C.2.2 Challenge 3: Beating the Curse of Dimensionality

The `level_scene` is a 22×22 grid (484 inputs). This massive input layer is the reason your network has 16,000 parameters, making it incredibly difficult for a simple evolutionary algorithm to optimize.

- **Task:** Reduce the size of the observation space.
- **Hint:** Mario does not need to know what is happening 11 blocks behind him, nor does he care about the skybox. Slice the `self.level_scene` numpy array to extract a smaller bounding box (e.g., 7×7) strictly representing the area immediately in front of and below Mario. Update `self.input_dim` accordingly.

References

[1] Julian Togelius, S. Karakovskiy, Jan Koutnik, and Jurgen Schmidhuber. Super mario evolution. pages 156 – 161, 10 2009.